

73. Custom Elements

Apni class se **custom HTML elements** bana sakte hain (apne methods, properties, events ke saath). Ek baar define hone par unhe built-in elements ki tarah use kar sakte ho.

HTML ki dictionary badi hai par infinite nahi: `<easy-tabs>`, `<sliding-carousel>`, `<beautiful-upload>` jaise tags nahi hain. Inhe khud define kar sakte hain.

2 tarah ke custom elements

1. **Autonomous** (swatantra): bilkul naye elements, abstract `HTMLElement` class extend karte hain.
2. **Customized built-in**: maujooda elements ka extension (jaise `HTMLButtonElement` par based customized button).

Class ka structure

Sab methods **optional** hain:

```
js
```

```

class MyElement extends HTMLElement {
  constructor() {
    super();
    // element ban gaya
  }

  connectedCallback() {
    // element document me add hone par (baar baar add/remove ho to kai baar)
  }

  disconnectedCallback() {
    // element document se hatne par
  }

  static get observedAttributes() {
    return [/* jin attributes ka badlav dekhna hai unke naam */];
  }

  attributeChangedCallback(name, oldValue, newValue) {
    // upar listed attributes me se koi badle to
  }

  adoptedCallback() {
    // element naye document me move ho (document.adoptNode, bahut kam use)
  }
}

customElements.define("my-element", MyElement); // register

```

Ab `<my-element>` tag ke liye `MyElement` ka instance banta hai. `document.createElement('my-element')` bhi kaam karta hai.

Naam me hyphen (-) zaruri hai (`my-element`, `super-button` sahi, `myelement` galat). Isse built-in aur custom elements ka naam conflict nahi hota.

Example: `<time-formatted>`

js

```

class TimeFormatted extends HTMLElement {
  connectedCallback() {
    let date = new Date(this.getAttribute('datetime') || Date.now());

    this.innerHTML = new Intl.DateTimeFormat("default", {
      year: this.getAttribute('year') || undefined,
      month: this.getAttribute('month') || undefined,
      day: this.getAttribute('day') || undefined,
      hour: this.getAttribute('hour') || undefined,
      minute: this.getAttribute('minute') || undefined,
      second: this.getAttribute('second') || undefined,
      timeZoneName: this.getAttribute('time-zone-name') || undefined,
    }).format(date);
  }
}

customElements.define("time-formatted", TimeFormatted);

```

html

```

<time-formatted datetime="2019-12-01"
  year="numeric" month="long" day="numeric"
  hour="numeric" minute="numeric" second="numeric"
  time-zone-name="short"></time-formatted>

```

Custom elements upgrade

`customElements.define` se pehle `<time-formatted>` mila to error nahi, bas wo abhi anjaan tag hai. Aise "undefined" elements ko CSS `:not(:defined)` se style kar sakte hain. `define` call hone par wo "**upgrade**" ho jaate hain (naya instance bana, `connectedCallback` chala).

Info ke liye:

- `customElements.get(name)`: class deta hai
- `customElements.whenDefined(name)`: promise jo define hone par resolve hota hai

Render `connectedCallback` me, `constructor` me nahi

`constructor` chalte waqt bahut jaldi hoti hai: element bana hai par browser ne abhi attributes assign nahi kiye, `getAttribute` `null` dega. Saath hi performance ke liye bhi behtar: kaam tabhi karo jab zarurat ho. `connectedCallback` tab chalta hai jab element **document ka hissa** ban jaye (sirf kisi ke child me append hona nahi). Isliye detached DOM bana ke rakh sakte ho, wo tab render hoga jab page me aayega.

Attributes observe karna

Upar wale element me render ke baad attribute badalne par kuch nahi hota, jo HTML element ke liye ajeeb hai (jaise `a.href` badalne par turant dikhta hai). Fix:

`observedAttributes` me attributes ki list do, unka badlav `attributeChangedCallback` me aayega (baaki attributes ke liye nahi, performance ke liye).

```
js

class TimeFormatted extends HTMLElement {
  render() {
    let date = new Date(this.getAttribute('datetime') || Date.now());
    this.innerHTML = new Intl.DateTimeFormat("default", { /* ... */ }).format(d
  }

  connectedCallback() {
    if (!this.rendered) {
      this.render();
      this.rendered = true;
    }
  }

  static get observedAttributes() {
    return ['datetime', 'year', 'month', 'day', 'hour', 'minute', 'second', 'ti
  }

  attributeChangedCallback(name, oldValue, newValue) {
    this.render();
  }
}

// live timer:
setInterval(() => elem.setAttribute('datetime', new Date()), 1000);
```

Rendering order

HTML parser DOM banate waqt elements ko ek ke baad ek process karta hai, **parents pehle, children baad me**. `<outer><inner></inner></outer>` me pehle `<outer>` create aur connect hota hai, phir `<inner>`.

Isse custom elements par asar: `connectedCallback` me `innerHTML` padho to **khaali** milta hai (children abhi bane hi nahi):

```
html
```

```
<user-info>John</user-info>
<script>
customElements.define('user-info', class extends HTMLElement {
  connectedCallback() { alert(this.innerHTML); } // khaali
});
</script>
```

Solutions:

- Data dena ho to **attributes** use karo (turant milte hain).
- Children chahiye to zero-delay (`setTimeout`) se access taalo:

```
js
connectedCallback() { setTimeout(() => alert(this.innerHTML)); } // John
```

Par ye perfect nahi: nested custom elements bhi (`setTimeout`) use karein to queue me lagte hain, isliye **outer** **pehle initialize** hota hai, **inner baad me**:

1. outer connected
2. inner connected
3. outer initialized
4. inner initialized

"Nested elements ready ho gaye" ka koi built-in callback nahi. Chahiye to khud: inner elements (`initialized`) jaise events dispatch karein aur outer sune.

Customized built-in elements

Naye elements (jaise `<time-formatted>`) ki koi semantics nahi hoti: search engines aur accessibility devices unhe nahi samajhte. Agar khaas button banana hai to `<button>` ki functionality kyun na reuse karein? Built-in classes se inherit karo.

1. Class extend karo:

```
js
class HelloButton extends HTMLButtonElement { /* ... */ }
```

2. (`define`) ka **teesra argument** do (`extends`), kyunki alag tags ek hi DOM class share kar sakte hain:

```
js
```

```
customElements.define('hello-button', HelloButton, {extends: 'button'});
```

3. HTML me normal `<button>` likho, bas `is="hello-button"` jodo:

html

```
<button is="hello-button">...</button>
```

Poora example:

js

```
class HelloButton extends HTMLButtonElement {
  constructor() {
    super();
    this.addEventListener('click', () => alert("Hello!"));
  }
}
customElements.define('hello-button', HelloButton, {extends: 'button'});
```

html

```
<button is="hello-button">Click me</button>
<button is="hello-button" disabled>Disabled</button>
```

Ye built-in button extend karta hai, isliye styles aur `disabled` jaise standard features bane rehte hain.

Summary

1. **Autonomous:** naye tags, `HTMLElement` extend (upar wali scheme).
2. **Customized built-in:** maujooda elements ka extension, `define` me extra argument aur HTML me `is="..."`.

Custom elements browsers me achhe se supported. Polyfill:

<https://github.com/webcomponents/polyfills/tree/master/packages/webcomponentsjs>

Task: `<live-timer>`

`<time-formatted>` ko andar use kare (duplicate nahi), har second update, aur har tick par `tick` custom event `event.detail` me current date ke saath. Dhyan: element document se hatne par (`disconnectedCallback`) `setInterval` **clear** karo, warna wo chalta rahega aur browser is element ki memory free nahi kar paayega. Class ke sab methods/properties naturally element ke bhi methods/properties hain (jaise `elem.date`).